



.NET CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. SETUP & CLI

Installation

```
dotnet new console -n MyApp
dotnet new webapi -n MyApi
dotnet add package Newtonsoft.Json
dotnet build / dotnet run / dotnet test
dotnet publish -c Release -o ./publish
dotnet tool install -g dotnet-ef
```

04. CLASSES & RECORDS

Workflow

```
public class User {
    public required string Name { get; init; }
    public int Age { get; set; }
}
// Record (value-equality, immutable):
public record Product(string Name, decimal Price);
var p2 = p1 with { Price = 19.99m }; // non-d...
```

02. BASIC SYNTAX

Architecture

```
var name = "Alice"; // type inferred
string? maybeNull = null; // nullable refere...
int[] numbers = { 1, 2, 3 };
(string First, int Age) tuple = ("Bob", 30);
Console.WriteLine($"Hello {name}");
using var stream = File.OpenRead("f.txt"); /...
```

05. COLLECTIONS

CRUD API

```
List<T> Dictionary<K,V> HashSet<T>
Queue<T> Stack<T> LinkedList<T>
IEnumerable<T> IReadOnlyList<T>
var list = new List<int> { 1, 2, 3 };
var dict = new Dictionary<string,int> { { "a",...
Span<T> Memory<T> zero-copy slices
```

03. DATA TYPES

YAML / Config

```
int long float double decimal bool char string
byte short uint ulong
DateTime DateTimeOffset TimeSpan
Guid (new Guid() or Guid.NewGuid())
object? anything = null;
dynamic d = "hello"; d = 42; // runtime type
record Person(string Name, int Age); // immut...
```

06. LINQ

Integration

```
var evens = numbers.Where(n => n % 2 == 0);
var names = users.Select(u => u.Name);
var sorted = users.OrderBy(u => u.Age).ThenBy...
var first = users.First(u => u.IsAdmin);
var groups = users.GroupBy(u => u.Role);
var flat = lists.SelectMany(l => l.Items);
users.Any() / .All() / .Count() / .Sum() / .A...
```



SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. ASYNC / AWAIT

Hardening

```
public async Task<User> GetUserAsync(int id)
{
    var user = await _repo.FindAsync(id);
    return user ?? throw new NotFoundException();
}
var result = await Task.WhenAll(t1, t2, t3);
await using var res = new AsyncResource();
var cts = new CancellationTokenSource(timeout...)
```

10. INTERFACES & INHERITANCE

Unit Tests

```
public interface IRepository<T> {
    Task<T?> FindAsync(int id);
    Task SaveAsync(T entity);
}
public class UserRepo : BaseRepo, IRepository...
sealed // prevent further inheritance
abstract // cannot be instantiated
```

08. EXCEPTION HANDLING

Observability

```
try {
    var result = await GetDataAsync();
} catch (HttpRequestException ex) when (ex.St...
throw new NotFoundException("Resource not fou...
} finally {
    logger.LogDebug("Cleanup");
}
throw new InvalidOperationException("msg");
```

11. NULLABLE REFERENCE TYPES

Commands

```
<Nullable>enable</Nullable> // in .csproj
string? nullableName = null;
string nonNull = nullableName ?? throw new Ex...
if (name is not null) { Console.Write(name.Le...
name?.Length // null-conditional
user!.Name // null-forgiving (I know it's ...
```

09. GENERICS

Optimization

```
public T Find<T>(IEnumerable<T> items, Func<T...
=> items.FirstOrDefault(predicate);
public class Repository<T> where T : class, I...
{ ... }
where T : struct // value type constraint
where T : new() // must have parameterless ...
```

12. COMMON GOTCHAS

Warning

async void = unobserved exceptions always use async Task
LINQ is lazy evaluate with .ToList()/ToArray() when needed
ConfigureAwait(false) in library code to avoid deadlocks
String comparison: StringComparer.OrdinalIgnoreCase
DateTime.Now is local; DateTime.UtcNow is UTC prefer UTC